

Sistema Multiagente de Recomendación de Moda con Embeddings Multimodales y Búsqueda Vectorial en PostgreSQL

Gustavo Daniel Gago Hernández

Diploma en Ciencia de Datos e Inteligencia Artificial Aplicada

Universidad Tecnológica (UTEC, Uruguay) – Tecnológico de Monterrey (México)

gusdani2@hotmail.com

Diciembre 2025

Resumen

Este trabajo presenta un sistema inteligente de recomendación de moda basado en embeddings multimodales, búsqueda vectorial y arquitectura multiagente, aplicado a catálogos reales de tiendas uruguayas. Los datos fueron provistos por la startup *Styles Designer* [9] e incluyen 16,660 productos tras la consolidación de cuatro archivos CSV obtenidos por *web scraping*.

El desarrollo siguió tres etapas diferenciadas: (1) **preparación de datos**, donde se aplicó limpieza, estandarización mediante LLMs y generación de embeddings bilingües con OpenFashion CLIP [2], resultando en 764 productos con imágenes válidas; (2) **entrenamiento supervisado**, con modelos CatBoost [8] para clasificación de categoría (17 clases) y género (6 clases), alcanzando F1-scores de 0.85 y 0.92 respectivamente mediante validación cruzada estratificada; y (3) **sistema en producción**, con arquitectura multiagente que integra búsqueda vectorial en PostgreSQL (pgvector), clasificación en tiempo real y orquestación con Azure OpenAI GPT-4.

Para la recuperación de productos se implementó búsqueda vectorial en PostgreSQL con la extensión pgvector [13], utilizando distancia coseno y una estrategia híbrida de fusión texto-imagen con técnicas de tipo *bagging* y *boosting* ligero a nivel de ranking. Sobre estos componentes se construyó un sistema multiagente coordinado por un orquestador que combina modelos de lenguaje grandes (Azure OpenAI GPT-4), CatBoost y consultas SQL con filtros semánticos, soportando entradas textuales y visuales.

Se discuten decisiones de diseño, resultados experimentales, limitaciones (tamaño y desbalanceo del dataset, estandarización incompleta de colores, ruido contextual en imágenes) y recomendaciones para futuras extensiones hacia generación de outfits completos y despliegue productivo.

Palabras clave: recomendación de moda, embeddings multimodales, búsqueda vectorial, CatBoost, sistemas multiagente, pgvector.

1. Introducción

En el comercio minorista de indumentaria, los catálogos digitales suelen estar fragmentados por tienda, con criterios heterogéneos para nombrar categorías, géneros, colores y descripciones de producto. Esto dificulta la integración de catálogos de múltiples comercios y la implementación de sistemas de recomendación que generen resultados coherentes y reutilizables.

En este trabajo se aborda el problema a partir de datos reales provistos por la startup *Styles Designer* [9], que recolecta catálogos de 35 tiendas uruguayas mediante *web scraping*. El objetivo es desarrollar un sistema que permita, a partir de una consulta en lenguaje natural o de una imagen de ejemplo, recuperar prendas similares de un catálogo unificado, preservando información relevante de categoría, género y estilo.

Para ello se propone una arquitectura que combina:

- Limpieza y estandarización de datos de múltiples fuentes.
- Generación de embeddings multimodales (texto e imagen) con OpenFashion CLIP [2].

- Clasificación supervisada con CatBoost [8] para categoría y género.
- Búsqueda vectorial con PostgreSQL + pgvector [13] basada en distancia coseno.
- Orquestación multiagente con modelos de lenguaje grandes (LLM) de Azure OpenAI.

El alcance se centra en la búsqueda inteligente de prendas individuales (no en la generación automática de outfits completos), priorizando la solidez en los componentes de representación, clasificación y recuperación.

2. Trabajos Relacionados

La recomendación de outfits ha sido abordada en la literatura principalmente desde el punto de vista del modelado de compatibilidad visual y semántica. FashionNet [12] propone una red neuronal profunda que combina extracción de características visuales y análisis de compatibilidad entre piezas, entrenada sobre conjuntos creados por usuarios en Polyvore. Hsiao y Grauman [5] abordan la creación de *capsule wardrobes* como un problema de

selección de subconjuntos, maximizando compatibilidad y versatilidad.

En el contexto de e-commerce, Vasileva et al. [10] introducen GORDN, un modelo que aprende compatibilidad entre productos a partir de ejemplos positivos y negativos, entrenado sobre conjuntos curados por estilistas (ASOS). Zhao et al. [11] proponen FashionRN, que modela la compatibilidad mediante redes relacionales sobre representaciones de prendas extraídas con DenseNet.

Estos trabajos se centran en la generación de outfits coherentes a partir de bases de productos ya consolidadas y relativamente limpias. En contraste, el presente trabajo se sitúa un paso antes: resolver la integración, estandarización y clasificación de catálogos heterogéneos de tiendas reales, y luego proporcionar una infraestructura de recuperación eficiente sobre esa base.

Adicionalmente, se apoya en el uso de embeddings multimodales específicos de moda, como OpenFashion CLIP [2], y en mecanismos de búsqueda vectorial sobre bases de datos relacionales mediante pgvector [13]. Los fundamentos de clasificación supervisada y problemas de alta dimensión siguen la línea clásica de Hastie et al. [4].

3. Metodología

El sistema se desarrolló en tres etapas secuenciales que se integran en producción (Figura 1):

1. **Preparación de datos:** Limpieza, estandarización con LLMs y generación de embeddings multimodales sobre el catálogo original.
2. **Entrenamiento supervisado:** Construcción de modelos CatBoost para clasificación de categoría y género.
3. **Sistema en producción:** Arquitectura multiagente que integra búsqueda vectorial, clasificación en tiempo real y orquestación con LLMs.

A continuación se detallan los componentes de cada etapa.

3.1. Descripción del Dataset

Styles Designer proporcionó cuatro archivos CSV: `products.csv`, `product_data.csv`, `categories.csv` y `shops.csv`, obtenidos por scraping de 35 tiendas uruguayas. Tras la unión relacional y filtrados de consistencia se consolidaron 16,660 productos de 31 tiendas.

De este conjunto, se aplicó un filtrado de calidad que retuvo únicamente productos con: (1) imagen válida disponible, (2) descripciones completas, y (3) categorización consistente tras validación manual. Este proceso resultó en un **dataset final de 764 productos** utilizados para generación de embeddings, entrenamiento de modelos y búsqueda vectorial.

Los productos se identifican mediante un `product_id` global. Las descripciones incluyen texto libre, categorías propias de la tienda, género (en distintos formatos) y URLs de imágenes. En el dataset original se detectaron problemas de:

- Variabilidad en los nombres de categoría y género.

- Valores faltantes o genéricos (p.ej. “Sin dato”).
- Presencia de productos fuera del dominio (golosinas, accesorios no vestibles).

3.2. Limpieza y Estandarización

Etapas 1: Preparación de Datos

La limpieza y estandarización se documentaron en el *ANEXO I*. Las tareas principales fueron:

1. Normalización de URLs y uniones entre tablas por `shop_url` y `product_id`.
2. Eliminación de productos irrelevantes (papelería, decoración, artículos deportivos, juguetes, etc.).
3. Unificación de género mediante una taxonomía global: {Dama, Caballero, Niño, Niña, Bebés, Unisex}.
4. Definición de categorías globales (34 etiquetas, p.ej. Remeras, Pantalones, Buzos, Vestidos, Calzado, Carteras, etc.).
5. Generación de una columna `text_for_embedding` que concatena nombre, descripción, categoría y género.

Para validar la estandarización se utilizaron modelos de lenguaje:

- **facebook/bart-large-mnli** [3], que mostró baja precisión en la clasificación de categorías globales, especialmente en casos ambiguos.
- **Azure OpenAI GPT-4o-mini** [7], que, con un *prompt* específico y temperatura 0, logró clasificaciones mucho más consistentes para categoría y género.

Sobre una muestra de 773 artículos con imagen, se etiquetaron 756 productos utilizando únicamente texto, descartando aquellos sin clasificación fiable.

3.3. Embeddings Multimodales

Se utilizaron embeddings de OpenFashion CLIP [2], un modelo multimodal especializado en moda. Para cada producto se generaron:

- **Embeddings de texto en español** (512 dimensiones) a partir de `text_for_embedding`.
- **Embeddings de texto en inglés** (512 dimensiones) mediante traducción automática con Google-Translator (`deep_translator`) y posterior codificación CLIP.
- **Embeddings de imagen** (512 dimensiones) a partir de la imagen principal, redimensionada a 224×224 y codificada desde Base64.

La tabla `product_embeddings` en PostgreSQL almacena los vectores `embedding`, `embedding_en` e `img_embedding`, junto con el texto de referencia en español e inglés.

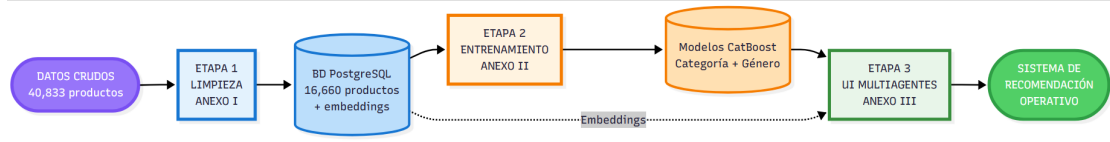


Figura 1: Pipeline integrado de 3 etapas del sistema. La base vectorial (Etapa 1) y los modelos entrenados (Etapa 2) se consumen en producción (Etapa 3).

3.4. Modelos de Clasificación (CatBoost)

Etapa 2: Entrenamiento de Modelos

Se entrenaron dos modelos de CatBoost [8]:

- **Modelo de categoría:** 17 clases con al menos 5 instancias.
- **Modelo de género:** 6 clases (Dama, Caballero, Niño, Niña, Bebés, Unisex).

Las características de entrada son vectores de 1024 dimensiones, obtenidos por concatenación de embeddings de texto en español e inglés. Para evitar *data leakage*, las columnas de texto utilizadas en embeddings no incluyen las etiquetas de categoría ni género generadas por los modelos de estandarización.

Debido al desbalanceo de clases, se aplicó:

- Filtrado de clases con menos de 5 muestras.
- Cálculo automático de *class weights* con `sklearn.utils.class_weight`.

La configuración típica fue:

- `iterations = 1000`, `depth = 6`, `learning_rate = 0.1`.
- `loss_function = MultiClass`.
- `early_stopping_rounds = 50`, con conjunto de validación.

Se utilizó validación cruzada estratificada (5 folds) para estimar capacidad de generalización.

3.5. Búsqueda Vectorial con pgvector

La recuperación de productos se basa en la extensión pgvector [13], que permite almacenar vectores y calcular similitudes mediante operadores especializados:

- `<=>`: distancia coseno estándar.
- `<#>`: distancia coseno negativa.

Dado que OpenFashionCLIP genera embeddings normalizados ($\|\mathbf{v}\| = 1$), ambos operadores se relacionan con la similitud coseno $\text{similitud_coseno} = \mathbf{u} \cdot \mathbf{v}$ de la siguiente forma:

$$\mathbf{u} \leq \mathbf{v} = 1 - \mathbf{u} \cdot \mathbf{v}, \quad \mathbf{u} \leq \# \mathbf{v} = -\mathbf{u} \cdot \mathbf{v}.$$

En este proyecto se utiliza el operador `<=>` y se define una similitud normalizada en el rango $[0, 1]$ como:

$$\text{similitud} = 1 - \frac{\mathbf{u} \leq \mathbf{v}}{2},$$

de modo que valores cercanos a 1 indican alta similitud y valores cercanos a 0 indican baja similitud.

Se implementaron cinco modos de búsqueda:

1. Texto $\rightarrow$ texto (ES).
2. Texto $\rightarrow$ texto (EN).
3. Imagen $\rightarrow$ imagen.
4. Texto $\rightarrow$ imagen (búsqueda cruzada).
5. Búsqueda híbrida con filtros por categoría/género.

El tamaño del catálogo con imagen (764 productos) permite usar búsqueda exhaustiva ($O(n)$) con latencias menores a 200 ms, por lo que no se implementaron índices HNSW [6] en esta fase; se discute su pertinencia para catálogos mayores a 5,000 productos.

3.6. Arquitectura Multiagente

Etapa 3: Sistema en Producción

Sobre los componentes de las etapas 1 y 2 (base vectorial en PostgreSQL y modelos CatBoost entrenados) se construyó un sistema multiagente coordinado, implementado en Python con Streamlit como frontend. Se definieron siete agentes principales:

- **Coordinador:** orquestador que decide el flujo de ejecución según la consulta.
- **AnalizadorConsulta:** extrae atributos estructurados (prenda, color, género, ocasión) de la consulta textual.
- **ImageAnalyzer:** usa GPT-4 Vision para describir imágenes de prendas.
- **ClasificadorCatBoost:** encapsula los modelos de categoría y género.
- **BuscadorEmbeddings:** ejecuta búsquedas vectoriales en PostgreSQL.
- **EspecialistaOutfits:** interpreta contexto de ocasión/estilo y genera prompts de búsqueda más ricos.
- **PresentadorResultados:** formatea los resultados en lenguaje natural y reordena productos.

Los *prompts* de los agentes LLM se almacenan en base de datos para permitir su edición sin despliegue de código. El coordinador puede operar en modo “inteligente” (con LLM) o modo *fallback* (sin LLM, con expresiones regulares y plantillas).

4. Resultados

4.1. Rendimiento de Clasificación

En el modelo de categoría, entrenado tras filtrar clases con menos de 5 muestras, se obtuvieron resultados representativos como los que se resumen en la Tabla 1.

Cuadro 1: Métricas del modelo CatBoost para categoría

Métrica	Valor	Comentario
Exactitud	0.87	Aciertos globales
Precisión	0.87	Predicciones positivas correctas
Recall	0.87	Cobertura de clases reales
F1-score	0.85	Media armónica P/R

El modelo de género alcanzó métricas más altas, como se muestra en la Tabla 2.

Cuadro 2: Métricas del modelo CatBoost para género

Métrica	Valor	Comentario
Exactitud	0.93	Alta tasa de aciertos
Precisión	0.93	Bajas falsas alarmas
Recall	0.93	Buen reconocimiento de clases
F1-score	0.92	Equilibrio P/R muy estable

Los F1-scores se obtuvieron por validación cruzada estratificada de 5 folds, lo que indica capacidad de generalización robusta pese al tamaño moderado del dataset. El detalle completo de métricas por clase, matrices de confusión y análisis de desviaciones estándar se documentó en el *ANEXO II*.

4.2. Evaluación de Embeddings de Imagen

Se evaluó la concatenación de los tres embeddings disponibles (texto ES, texto EN e imagen) como input a CatBoost. Aunque esto mejoraba en algunos casos la recuperación de similitud, se observó un problema de *ruido contextual*: el modelo tendía a sobreajustar a elementos secundarios de la imagen (por ejemplo, un buzo llamativo usado por la modelo), produciendo predicciones erróneas de categoría o color.

Debido a este efecto y a la alta variabilidad en las condiciones de las fotos (fondos, poses, múltiples prendas), se decidió excluir el `img_embedding` del entrenamiento y utilizarlo exclusivamente para búsqueda visual en la base.

4.3. Búsqueda Vectorial e Integración Híbrida

Las pruebas de búsqueda con consultas textuales simples (p. ej. “remera azul manga corta para hombre”) mostraron que:

- El modelo de género identificaba correctamente el género como Caballero.
- El modelo de categoría asignaba Remeras o Buzos según la descripción.

- La búsqueda vectorial recuperaba productos razonablemente similares, aunque con cierta variabilidad en colores debido a la falta de estandarización de esa variable.

En el caso de consultas por imagen, cuando se cargaba una fotografía presente en el catálogo, la búsqueda por `img_embedding` recuperaba como primer resultado la misma prenda, lo que confirma la consistencia interna de los embeddings.

La búsqueda híbrida texto-imagen, combinando similitud de texto e imagen con una lógica de *bagging* (selección de la máxima similitud entre las tres vistas) y un *boost* adicional para productos que aparecen tanto en la búsqueda de texto como de imagen, mostró mejoras en precisión *top-k* frente a los modos unimodales, de acuerdo con análisis internos inspirados en [1].

4.4. Desempeño del Sistema Multiagente

En el frontend desarrollado con Streamlit, la interacción típica del usuario consiste en:

1. Ingresar una consulta textual (español o inglés) y/o adjuntar una imagen.
2. El coordinador decide invocar agentes de análisis de consulta, clasificación, búsqueda y presentación.
3. Se devuelven al usuario las prendas más similares, con indicación de la similitud y de la predicción de categoría y género.

En las pruebas cualitativas se observaron:

- Respuestas consistentes cuando el usuario combina contexto de ocasión (p. ej. “para fiesta”) con descripción de prenda.
- Capacidad de detectar consultas en inglés y adaptar el idioma de la respuesta, aunque la detección de idioma podría mejorarse.
- Manejo adecuado de imágenes no válidas (no prendas), que son rechazadas por el agente de visión.

5. Discusión

5.1. Interpretación de Resultados

El pipeline completo permitió pasar desde datos altamente heterogéneos y ruidosos a un sistema funcional capaz de:

- Estandarizar géneros y categorías globales de productos de múltiples tiendas.
- Representar prendas mediante embeddings multimodales consistentes.
- Clasificar con buena precisión la categoría y el género de nuevas descripciones.
- Recuperar productos similares mediante búsqueda vectorial eficiente.

- Orquestar estos componentes en una arquitectura multiagente flexible.

La reorientación del alcance, desde la generación de outfits completos hacia una búsqueda robusta de prendas individuales, permitió profundizar en aspectos de limpieza de datos, diseño de embeddings y evaluación de modelos en un contexto realista pero acotado.

5.2. Limitaciones

Entre las principales limitaciones se destacan:

- **Tamaño y desbalanceo del dataset:** el número de productos con imagen utilizable (764) y la distribución desigual entre categorías obligaron a filtrar clases minoritarias.
- **Color no estandarizado:** las etiquetas de color en las tiendas carecían de coherencia (incluyendo descripciones libres, códigos internos y fragmentos de URL), lo que impidió entrenar un clasificador de color fiable.
- **Ruido contextual en imágenes:** el embedding visual captura elementos irrelevantes (fondo, cabello, prendas secundarias), lo que dificulta su uso directo para clasificación.
- **Dependencia de LLM externos:** la arquitectura actual se apoya en servicios de Azure OpenAI, lo que implica costos y consideraciones de privacidad. El modo *fallback* basado en reglas cubre sólo casos simples.
- **Falta de contexto estacional y demográfico:** el sistema no incorpora aún información sobre estación del año, clima o rango de edad del usuario, limitando la personalización.

5.3. Consideraciones Éticas

El uso de *web scraping* para obtener catálogos requiere, en un entorno productivo, acuerdos explícitos con las tiendas y el respeto a los términos de servicio de cada sitio. Asimismo, el manejo de consultas de usuarios y de imágenes personales demanda cumplimiento de normativas de protección de datos y anonimización previa al envío a APIs externas.

Por otra parte, los modelos utilizados pueden reflejar sesgos culturales y de género presentes tanto en los datos de entrenamiento de CLIP como en el propio catálogo, por lo que se recomienda auditorías periódicas de equidad y mecanismos de corrección.

6. Conclusiones

Este trabajo implementó un sistema de recomendación de moda basado en embeddings multimodales, clasificación supervisada y búsqueda vectorial, integrado en una arquitectura multiagente coordinada. A partir de datos reales y ruidosos de tiendas uruguayas, se logró:

- Estandarizar categorías y géneros mediante LLMs y reglas.

- Entrenar modelos de CatBoost con F1-scores superiores al 0.85 (categoría) y 0.92 (género).
- Implementar búsqueda vectorial eficiente sobre PostgreSQL con pgvector, en modalidades texto, imagen e híbrida.
- Construir un frontend funcional con agentes especializados y orquestación dinámica de flujos.

Como líneas de trabajo futuro se identifican:

- Ampliar el dataset y reentrenar modelos con mayor cobertura de categorías.
- Incorporar clasificación de color robusta, posiblemente mediante modelos específicos y curación manual.
- Explorar índices HNSW para escalar a catálogos de decenas de miles de productos.
- Extender la arquitectura para generación de outfits completos, integrando modelos de compatibilidad inspirados en [12, 10, 11].
- Evaluar despliegues con modelos locales para reducir dependencia de proveedores externos y mejorar la privacidad.

Más allá de los resultados cuantitativos, el proyecto permitió validar una arquitectura híbrida que combina LLMs, modelos clásicos de machine learning y bases de datos vectoriales, aplicable a otros dominios con datos no estructurados y catálogos heterogéneos.

Referencias

- [1] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, August 1996.
- [2] Giuseppe Cartella, Alberto Baldrati, Davide Morelli, Marcella Cornia, Marco Bertini, and Rita Cucchiara. OpenFashionCLIP: Vision-and-Language Contrastive Learning with Open-Source Fashion Data. In *Proceedings of the International Conference on Image Analysis and Processing*, 2023.
- [3] Facebook AI. Modelo de Categorización de Facebook. <https://huggingface.co/facebook/bart-large-mnli>, 2024. modelo tipo BART (Bidirectional and Auto-Regressive Transformer) entrenado por Facebook AI, pensado para la tarea de Reconocimiento de Interferencia Natural (NLI).
- [4] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2 edition, 2009.
- [5] Wei-Lin Hsiao and Kristen Grauman. Creating capsule wardrobes from fashion images, 2017.
- [6] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *arXiv preprint arXiv:1603.09320*, 2016.

- [7] Microsoft Azure. Azure openai service. <https://azure.microsoft.com/en-us/products/cognitive-services/openai-service>, 2024. Facilitado por Styles Designer para este trabajo.
- [8] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648, 2018.
- [9] Styles Designer. Startup con financiación ANII Uruguay. <https://stylesdesigner.com/>, 2024. Todos los datos de este trabajo son propiedad de Styles Designer.
- [10] Miglena Vasileva, Bryan A. Plummer, Karan Dussad, Rahul Rajpal, Shreya Kumar, and David Forsyth. Fashion outfit generation for e-commerce, 2019.
- [11] Ruizhongtai Zhao, Ke Cheng, and Jiashi Lu. Fashion recommendation and compatibility prediction using relational network, 2020.
- [12] Ruizhongtai Zhao, Min Wu, Ke Cheng, Yuncheng Jiang, and Jiashi Lu. Fashionnet: Personalized outfit recommendation with deep neural network, 2018.
- [13] Zilliz and Open-Source Contributors. pgvector: Open-source vector similarity search for postgresql. <https://github.com/pgvector/pgvector>, 2023. Extensión para almacenar y buscar embeddings mediante índices vectoriales como IVFFlat y HNSW.